Plan de trabajo (8 semanas)

Integración de un asistente docente de IA en Moodle

Planificación realista de PoC desde cero, priorizando integración Moodle, frontend y pipeline RAG/embeddings, con backend mínimo centralizado.

Objetivo final	Disponer al final de la semana 8 de una PoC funcional donde Moodle envía el contexto del usuario y del curso, el sistema recupera material docente mediante RAG y el asistente responde de forma diferenciada para alumno y profesor.
Supuesto de reparto	Adolfo centraliza arquitectura, backend mínimo, IA/MLOps, despliegue y evaluación. Manuel, Marcos y Andrés se orientan sobre todo a integración Moodle, frontend y pipeline de embeddings/contenidos.

1. Reparto de roles propuesto

Adolfo — Líder técnico de IA/MLOps y supervisor

- Arquitectura global, definición de stack y decisiones técnicas no triviales.
- Backend mínimo en FastAPI, orquestación del flujo y control de acceso por rol/curso.
- Docker Compose, variables de entorno, despliegue local reproducible y revisión de PRs.
- Diseño del RAG, prompts, evaluación y cierre de demo.

Manuel — Integración Moodle

- Bloque, plugin o widget embebido en Moodle.
- Captura y envío de user_id, course_id y rol.
- Validación del flujo dentro del LMS.

Marcos — Frontend / UX del chat

- Interfaz del chat, estados de carga/error y experiencia de uso.
- Conexión frontend-backend sobre contratos ya definidos.
- Pulido visual y soporte a la demo.

Andrés — Contenidos, chunking y embeddings

- Extracción de materiales, limpieza de texto y metadatos.
- Chunking, embeddings y carga en base vectorial.
- Pruebas de recuperación y dataset de evaluación.

2. Reglas de alcance para mantener el proyecto realista

- Trabajar al inicio con 1 o 2 asignaturas piloto, no con toda la universidad.
- Incluir únicamente dos perfiles de uso en la PoC: alumno y profesor.
- Arrancar con PDFs, apuntes y un subconjunto controlado de materiales.
- Evitar en esta fase sincronización compleja en tiempo real, analítica avanzada y despliegue cloud serio.
- Priorizar un flujo extremo a extremo que funcione sobre un backend pequeño: Moodle → API → recuperación RAG → LLM → respuesta.

3. Planning semanal detallado

A partir de este punto, se recomienda tratar cada semana como un mini sprint cerrado, con un entregable visible al final de la misma.

4. Criterios de éxito de la PoC al final de la semana 8

- El usuario accede al chat desde Moodle y el sistema conoce curso y rol.
- La respuesta se apoya en materiales de la asignatura piloto recuperados vía RAG.
- Alumno y profesor reciben comportamientos diferenciados.
- La solución puede levantarse en local mediante un procedimiento documentado.
- Existe una demo estable y defendible, aunque todavía no sea un sistema de producción.

Semana 1 — Arranque técnico y congelación del alcance

Objetivo de la semana: Alinear al equipo, dejar cerrado el reparto y disponer de una base ejecutable mínima en local.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Cerrar la arquitectura y dejar preparado el esqueleto técnico.	• Definir stack final, estructura del repositorio y normas de trabajo. • Montar FastAPI mínima, Docker Compose base y variables de entorno. • Redactar backlog inicial y criterios de aceptación de la PoC.	Repositorio listo, entorno local reproducible y backlog priorizado.
Manuel	Validar la vía más simple para incrustar el chat en Moodle.	• Analizar bloque, plugin o widget embebido. • Hacer una prueba visual simple dentro del LMS.	Prueba de integración visible en Moodle.
Marcos	Diseñar la primera versión de la interfaz del chat.	• Crear una maqueta simple con entrada, salida y mensajes de estado. • Dejar preparada la estructura visual para conectar con API.	Maqueta navegable del chat.
Andrés	Preparar la base del pipeline de contenidos.	• Reunir materiales de una asignatura piloto. • Probar extracción de texto con varios PDFs y definir esquema de metadatos.	Muestra de documentos procesados y metadatos acordados.

Semana 2 — Primer flujo extremo a extremo sin IA real

Objetivo de la semana: Conseguir que Moodle, interfaz y backend se hablen entre sí con datos reales o simulados.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Dejar el endpoint /chat operativo con respuesta simulada.	• Definir el contrato JSON del chat. • Implementar /health y /chat con respuesta dummy. • Configurar logging técnico básico.	Flujo funcional Moodle/frontend → backend.
Manuel	Enviar al backend el contexto del usuario desde Moodle.	• Capturar user_id, course_id y rol. • Validar que el LMS pasa esos datos al punto de entrada del chat.	Payload real llegando al backend.
Marcos	Conectar la interfaz con el endpoint dummy.	• Consumir /chat desde el frontend. • Gestionar loading, errores y respuesta simple.	Chat conectado a una respuesta simulada.
Andrés	Definir chunking y metadatos con criterio.	• Acordar campos mínimos: course_id, source, chunk_id, rol si procede. • Hacer primeras particiones de texto manuales o semi-automáticas.	Esquema estable de chunking y metadatos.

Semana 3 — Pipeline de embeddings y base vectorial

Objetivo de la semana: Poblar la base vectorial con una asignatura piloto y dejar el flujo de ingesta razonablemente limpio.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Fijar la estrategia inicial de retrieval.	• Elegir vector DB, modelo de embeddings y parámetros base. • Definir tamaño de chunk, solape y top-k inicial.	Criterios técnicos de retrieval cerrados.
Manuel	Mejorar la integración Moodle con contexto visible.	• Mostrar curso/rol activo en el chat. • Resolver fricciones de navegación dentro del LMS.	Integración más usable y entendible.
Marcos	Pulir la UX del chat.	• Mejorar mensajes, formato de respuesta y comportamiento de errores. • Preparar componentes para mostrar referencias o avisos si luego se añaden.	Interfaz más clara y estable.
Andrés	Implementar la ingesta real.	• Extracción de materiales, limpieza de texto, chunking y embeddings. • Carga en base vectorial con metadatos por curso.	Asignatura piloto indexada en la vector DB.

Semana 4 — Primer RAG extremo a extremo

Objetivo de la semana: Conseguir respuestas basadas en materiales del curso, no respuestas genéricas.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Integrar recuperación + prompt + llamada al modelo.	• Conectar /chat con la base vectorial. • Construir prompt con contexto recuperado. • Filtrar por course_id antes de llamar al LLM.	Chat RAG funcional sobre una asignatura.
Manuel	Probar el flujo dentro de Moodle con escenarios reales.	• Verificar que el contexto del curso es correcto. • Identificar fallos de integración o sesiones.	Lista de incidencias de integración y correcciones.
Marcos	Mejorar la representación de respuestas y errores.	• Mostrar tiempos de espera de forma amigable. • Cuidar scroll, envío y lectura continua.	Frontend preparado para demo interna.
Andrés	Revisar la calidad de lo recuperado.	• Detectar chunks demasiado largos, vacíos o duplicados. • Corregir parsing y metadatos cuando haga falta.	Base vectorial más limpia y retrieval más útil.

Semana 5 — Personalización por rol y control de acceso

Objetivo de la semana: Separar claramente el comportamiento de alumno y profesor y evitar fugas de contexto.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Diseñar el comportamiento diferencial y aplicar RBAC mínimo.	• Definir prompts diferenciados para alumno y profesor. • Limitar acceso según rol y curso. • Revisar cómo se gestionan casos fuera de contexto.	Reglas funcionales de acceso y prompts diferenciados.
Manuel	Verificar qué rol aporta Moodle y cómo exponerlo bien.	• Asegurar que el rol llega correctamente al backend. • Probar combinaciones alumno/profesor en cursos distintos.	Validación del flujo de roles desde Moodle.
Marcos	Ajustar la interfaz a la personalización.	• Mostrar avisos o etiquetas del modo activo. • Refinar el tono visual para que el asistente se perciba como herramienta docente.	UX coherente con la diferenciación por rol.
Andrés	Refinar metadatos y pruebas por perfil.	• Etiquetar mejor documentos o fuentes por curso y tipo. • Preparar preguntas distintas para alumno y profesor.	Conjunto de pruebas por rol.

Semana 6 — Evaluación técnica y estabilización

Objetivo de la semana: Dejar de probar solo a ojo y medir si la PoC responde con calidad suficiente.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Liderar la evaluación y ajustar parámetros.	• Crear batería de preguntas reales y criterios de éxito. • Ajustar top-k, prompts y filtros según resultados. • Revisar tiempos de respuesta y fallos frecuentes.	Matriz de evaluación con conclusiones y ajustes.
Manuel	Cerrar bugs de integración Moodle.	• Resolver incidencias abiertas de sesión, carga y navegación. • Comprobar estabilidad tras cambios del backend.	Integración estable para pruebas continuadas.
Marcos	Pulir bugs de experiencia de usuario.	• Corregir errores de envío, renderizado y estados. • Homogeneizar mensajes y comportamiento del chat.	Frontend listo para uso prolongado en demo.
Andrés	Formalizar el dataset de evaluación.	• Incluir preguntas fáciles, ambiguas y fuera de temario. • Registrar ejemplos donde la recuperación falle.	Dataset técnico de evaluación y recuperación.

Semana 7 — Cierre técnico de la PoC

Objetivo de la semana: Congelar la solución, documentarla y evitar cambios de última hora que rompan la demo.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Congelar arquitectura y documentar el arranque.	• Cerrar la API mínima y revisar Docker Compose. • Documentar instalación, variables de entorno y flujo de ejecución. • Pasar revisión global del repositorio.	Versión candidata a demo con documentación de despliegue.
Manuel	Documentar la parte de Moodle.	• Explicar cómo se integra el chat en el LMS. • Dejar instrucciones claras para replicar el montaje.	Guía breve de integración Moodle.
Marcos	Pulido final del chat.	• Revisar textos, estados visuales y coherencia general. • Corregir detalles que afecten a la percepción de calidad.	Interfaz final para presentación.
Andrés	Cerrar el pipeline de ingesta.	• Limpiar scripts y documentar pasos. • Asegurar que una nueva asignatura piloto podría cargarse sin rehacer todo.	Script de ingesta reproducible y documentado.

Semana 8 — Ensayo, demo y entrega

Objetivo de la semana: Presentar una PoC defendible, con narrativa clara y límites bien explicados.

Responsable	Objetivo semanal	Tareas clave	Entregable
Adolfo	Liderar la demo y el mensaje técnico.	• Preparar guion: problema, arquitectura, flujo, valor diferencial y límites. • Ensayar casos de uso de alumno y profesor. • Dejar resueltas incidencias críticas de última hora.	Demo final y discurso técnico coherente.
Manuel	Soporte a la demo desde Moodle.	• Asegurar que el acceso al chat funciona durante la presentación. • Cubrir navegación y contexto de usuario.	Parte LMS estable durante la demo.
Marcos	Soporte a la demo desde la interfaz.	• Controlar la experiencia del chat y posibles errores visibles. • Apoyar la presentación de usabilidad.	Flujo visual limpio y sin sorpresas.
Andrés	Soporte a la demo desde el RAG.	• Explicar de forma sencilla extracción, chunking y embeddings. • Preparar ejemplos de recuperación correcta.	Apoyo técnico a la explicación del valor del RAG.

5. Anotaciones de uso del repositorio en GitHub

• Mantener main como rama estable y presentable, sin cambios que rompan la demo.

• Usar develop como rama de integración antes de pasar cambios a main.

• Crear ramas por tarea y no por persona; por ejemplo: feat/moodle-context, feat/chat-ui, feat/rag-ingestion o fix/pdf-parser.

• Abrir siempre Pull Request hacia develop y evitar pushes directos a main y develop.

• Hacer commits pequeños y frecuentes, con mensajes claros y orientados a la tarea.

• Esquema en árbol del flujo de GitHub:

```
main
└─── rama estable del proyecto
    └─── solo contiene versiones revisadas y preparadas para demo

develop
└─── rama de integración del equipo
    ├─── feat/moodle-context-auth
    │    └─── integración de usuario, curso y rol desde Moodle
    ├─── feat/chat-ui
    │    └─── interfaz del chat y experiencia de usuario
    ├─── feat/rag-ingestion
    │    └─── extracción de documentos, chunking y embeddings
    ├─── feat/vector-search
    │    └─── búsqueda semántica en base vectorial
    ├─── feat/role-based-prompts
    │    └─── personalización por alumno/profesor
    ├─── fix/course-filter
    │    └─── corrección de filtrado por asignatura
    └─── docs/setup-local
         └─── documentación de instalación y despliegue local
```

• Esquema simple del flujo de trabajo:

1. Crear una issue

   ↓

2. Crear rama desde develop

   ↓

3. Desarrollar la tarea

   ↓

4. Hacer commits pequeños y claros

   ↓

5. Abrir Pull Request hacia develop

   ↓

6. Revisar y probar

   ↓

7. Fusionar en develop

   ↓

8. Cuando develop esté estable, fusionar en main

• Reparto de equipo:

```
main
 └─── estable / demo final

develop
 ├─── ramas de Adolfo
 │    ├─── feat/backend-chat-orchestrator
 │    ├─── feat/rbac-filtering
 │    └─── chore/docker-compose
 ├─── ramas de Manuel
 │    ├─── feat/moodle-plugin-context
 │    └─── feat/moodle-chat-embed
 ├─── ramas de Marcos
 │    ├─── feat/chat-ui
 │    └─── feat/chat-loading-errors
 └─── ramas de Andrés
      ├─── feat/rag-ingestion
      ├─── feat/embeddings-pipeline
      └─── feat/vector-metadata
```